

Compte rendu

Projet IoT



Mathieu Maisonnète – Chloé Larroze



Responsables académiques : E.Karbouche, A.Marques

Table des matières

1. Introduction.....	3
2. La préservation de la qualité de l'air dans des stations de métro confinées.....	4
2.1. Suivi des paramètres environnementaux.....	4
2.1.1. La nécessité d'un suivi en temps réel.....	4
2.1.2. Les paramètres à observer.....	4
2.2. Evacuation de l'air vicié.....	5
2.2.1. Les méthodes existantes.....	5
2.2.2. L'intérêt d'un ventilateur isolé.....	5
2.2.3. Un système d'évacuation de l'air contrôlé intelligemment.....	6
3. L'environnement difficile d'une station de métro.....	6
3.1. Intégration du système aux infrastructures et alimentation.....	6
3.2. Communiquer avec le système.....	7
3.2.1. La communication Lora.....	7
3.2.2. Communiquer sous la terre.....	7
4. Conception du système.....	8
4.1. Synoptique.....	8
4.2. Description matérielle du module.....	9
4.3. Mise en place de la communication LoRa.....	10
4.3.1. Code.....	10
4.3.2. Antenne.....	10
4.4. Intégration de la carte air monitoring.....	11
4.5. Interface de contrôle et visualisation via Datacake.....	11
4.6. Downlink et intégration d'un ventilateur.....	12
4.6.1. Mise en place du moteur.....	12
4.6.2. Contrôle du ventilateur.....	12
5. Test du système.....	13
5.1. Conception d'une maquette.....	13
5.2. Interprétation des résultats.....	14
5.2.1. Analyse perf.....	14
5.2.2. Actions.....	14
5.3. Limites observées.....	15
5.3.1. Consommation énergétique.....	15
5.3.2. Qualité du réseau Lora.....	15
5.4. Améliorations potentielles.....	15
5.4.1. Techniques.....	15
5.4.2. Régulation du trafic et des passagers.....	16
6. Conclusion.....	16
Bibliographie et sources.....	17
Annexes.....	18
Types de trames CayenneLPP.....	18
Contrôle automatique du moteur.....	18
Encodeur du Downlink de contrôle du moteur.....	18
Encodeur du Downlink de changement des paramètres.....	19

1. Introduction

Une étude menée pour l'émission *Vert de Rage* de France Télévisions a révélé que certaines stations du métro parisien présentent des niveaux de pollution aux particules fines jusqu'à cinq fois supérieurs aux recommandations de l'OMS. Bien que la RATP ait déjà mis en place certaines mesures, comme des systèmes de renouvellement de l'air et l'utilisation de garnitures de frein spécifiques, ces efforts restent insuffisants pour garantir un air sain aux usagers. Un suivi plus précis et en temps réel permettrait non seulement de mieux évaluer l'impact des solutions existantes, mais aussi d'adapter la gestion du trafic et des infrastructures pour limiter l'exposition des voyageurs à la pollution.

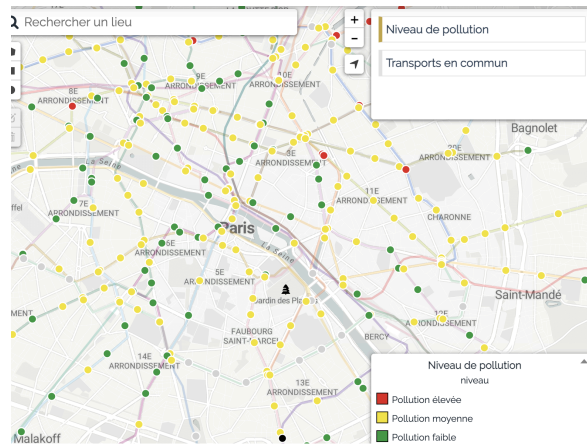


figure 1 : Qualité de l'air dans le réseau de transport transilien [3]

Dans ce contexte, ce projet vise à développer un POC pour un système IoT permettant de surveiller et d'analyser en temps réel les paramètres environnementaux dans le métro parisien. Le système proposé intègre plusieurs capteurs de mesure, capables de détecter des indicateurs comme les particules fines, les niveaux de CO₂, l'humidité et la température. Ces données sont ensuite transmises via une liaison sans fil LoRa, permettant une portée longue et à basse consommation énergétique.

Nous nous demanderons à cet effet **comment limiter l'exposition des usagers du métro à la pollution pour préserver leur santé ?** Pour cela, nous nous donnerons 4 objectifs principaux:

- **Intégration des capteurs** : Sélectionner, calibrer et assurer la mesure des paramètres environnementaux.
- **Communication sans fil (LoRa)** : Configurer une transmission des données en milieu souterrain.
- **Ventilation** : Adapter le renouvellement de l'air selon la pollution mesurée.
- **Exploitation des données** : Visualiser et ajuster le système en temps réel via Datacake.

Le développement du projet suivra donc plusieurs étapes, allant de la conception du prototype à l'analyse des résultats obtenus, en passant par l'intégration des capteurs, la mise en place de la communication LoRa et l'implémentation d'un contrôle intelligent du renouvellement de l'air. Dans ce rapport nous allons détailler les différentes contraintes de notre système et les données que nous voulons mesurer.

2. La préservation de la qualité de l'air dans des stations de métro confinées

2.1. Suivi des paramètres environnementaux

2.1.1. La nécessité d'un suivi en temps réel

La pollution de l'air dans le métro est un phénomène "dynamique", influencé par de nombreux facteurs comme le passage des métros, l'afflux de passagers, différent selon les heures de la journée, la ventilation ou encore l'aération naturelle des tunnels selon les stations. Les niveaux de particules fines et de CO₂ peuvent ainsi varier fortement au cours d'une même journée, rendant une surveillance en temps réel essentielle pour détecter rapidement des pics de pollution et agir en conséquence.

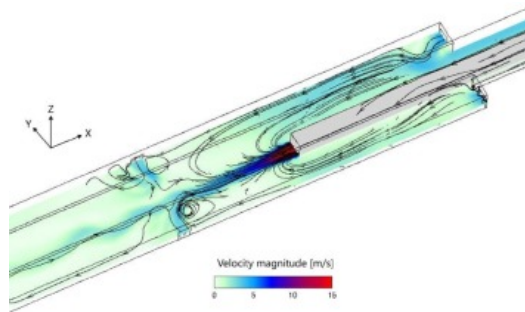


figure 2 : Flux d'air dans une station de métro

Un suivi en continu permettrait d'identifier les périodes critiques où la qualité de l'air se détériore, notamment aux heures de pointe ou en cas de dysfonctionnement des systèmes de ventilation. En disposant de mesures actualisées, il devient possible d'ajuster dynamiquement la circulation de l'air via des dispositifs comme des ventilateurs contrôlés automatiquement ou d'informer les voyageurs de la situation en temps réel via les applications de mobilité.

2.1.2. Les paramètres à observer

Pour être le plus efficace, nous nous devons de cibler les paramètres que nous désirons traiter.

En premier lieu, nous relevons la température de la station, qui peut non seulement nuire au confort des usagers mais aussi, dans certains cas, avoir un impact sur leur santé, notamment pour les personnes vulnérables. De plus, la température influence directement la formation de certains polluants comme l'ozone, qui peut s'accumuler et dégrader la qualité de l'air.

L'humidité relative est un autre facteur déterminant pour le bien-être des voyageurs. Une humidité trop élevée, combinée à une température élevée, peut rendre l'air difficilement respirable et favoriser la sensation d'inconfort. À l'inverse, un air trop sec peut provoquer des irritations des voies respiratoires.

Enfin, une concentration excessive en CO₂ peut engendrer une sensation de malaise, de fatigue et même des maux de tête chez les usagers. Pour surveiller ce paramètre, nous utilisons l'indice AQI (Indoor Air Quality), qui fournit une estimation globale de la qualité de l'air intérieur en prenant en compte divers polluants gazeux, suivant la table ci-dessous.

IAQ Index	Air Quality	Impact (long-term exposure)	Suggested action
0 – 50	Excellent	Pure air; best for well-being	No measures needed
51 – 100	Good	No irritation or impact on well-being	No measures needed
101 – 150	Lightly polluted	Reduction of well-being possible	Ventilation suggested
151 – 200	Moderately polluted	More significant irritation possible	Increase ventilation with clean air
201 – 250 ^a	Heavily polluted	Exposition might lead to effects like headache depending on type of VOCs	optimize ventilation
251 – 350	Severely polluted	More severe health issue possible if harmful VOC present	Contamination should be identified if level is reached even w/o presence of people; maximize ventilation & reduce attendance
> 351	Extremely polluted	Headaches, additional neurotoxic effects possible	Contamination needs to be identified; avoid presence in room and maximize ventilation

figure 3 : Table AQI, datasheet BME680

Nous relèverons également la pression, bien que cette dernière n'ait que peu d'impact sur la qualité de l'air du métro.

2.2. Evacuation de l'air vicié

2.2.1. Les méthodes existantes

L'évacuation de l'air vicié repose généralement sur des systèmes de ventilation centralisés. La ventilation forcée repose sur l'utilisation de ventilateurs industriels situés dans des puits d'extraction qui permettent de renouveler l'air du réseau. Ces ventilateurs sont centralisés et l'air est extrait dans plusieurs couloirs du métro sur une longue distance grâce à des ventilateurs puissants. L'air est ensuite évacué vers l'extérieur via des bouches d'aération. A Paris, la RATP, service de transport francilien, utilise notamment des bouches d'aération géantes cachées derrière des immeubles factices comme sur la photo ci-contre.

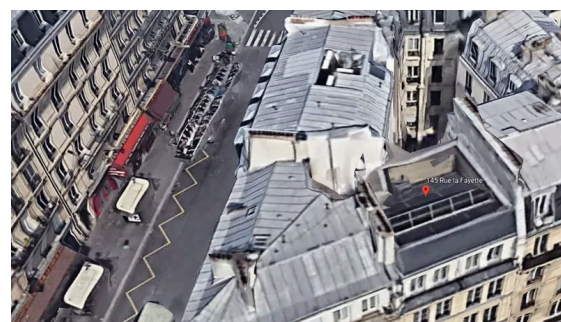


figure 4 : Bouche de ventilation dans immeuble factice

Des relevés effectués sur les toits montrent un indice de la qualité de l'air moyen à 70 ce qui correspond à un air vicié. Cette solution amène donc à une pollution importante de l'air extérieur.

Une autre méthode, la ventilation naturelle, exploite les courants d'air générés par le mouvement des trains, connus sous le nom d'effet piston, pour favoriser le renouvellement de l'air. Enfin, les systèmes hybrides combinent ventilation naturelle et assistée afin d'optimiser la consommation énergétique en adaptant la ventilation aux besoins en temps réel.

Bien que fonctionnelles, ces solutions présentent des limites en termes de réactivité et de consommation énergétique. De plus leur temps de mise en action est élevé, il est donc intéressant de réfléchir à une solution plus flexible et permettant d'agir brièvement mais efficacement pour protéger les usagers.

2.2.2. L'intérêt d'un ventilateur isolé

L'implantation de ventilateurs isolés permet une gestion plus fine de la qualité de l'air dans des zones spécifiques via une intervention ciblée, en permettant d'agir localement sur des zones où la pollution est particulièrement élevée. On peut alors évacuer l'air via des évacuations dans la rue ou en hauteur de largeur plus réduites et sans nécessiter des

infrastructures géantes. De plus, ils offrent une flexibilité accrue en pouvant être activés à la demande en fonction des mesures des capteurs environnementaux. En optimisant leur fonctionnement, ces ventilateurs limitent la consommation d'énergie aux besoins réels, améliorant ainsi leur efficacité énergétique.



figure 5: Ventilation RATP

Un avantage majeur du ventilateur isolé est qu'il peut être intégré aux infrastructures en place sans nécessiter de modifications lourdes. Cette intégration simplifiée repose sur l'utilisation des réseaux électriques et de communication déjà existants dans les tunnels du métro, ce qui facilite grandement l'installation. De plus, son fonctionnement indépendant des systèmes centralisés réduit la complexité des interventions de maintenance. Cette indépendance permet aussi de limiter les coûts d'installation et d'entretien par rapport à une refonte complète des systèmes de ventilation.

Nous souhaitons donc prototyper un système utilisant un ventilateur de taille réduite afin d'évacuer l'air vicié vers l'extérieur de la station et cela uniquement lorsque c'est nécessaire.

2.2.3. Un système d'évacuation de l'air contrôlé intelligemment

Un ventilateur électrique est alimenté via un moteur, ce qui en fait un système énergivore et bruyant. La vitesse du moteur est proportionnelle à la puissance de celui-ci. Il est donc intéressant de limiter la vitesse de rotation afin de réduire la consommation énergétique mais également le bruit. De plus, un ventilateur tournant en permanence va s'user prématurément nécessitant alors des maintenances régulières. Enfin, une extraction trop rapide de l'air pourrait perturber l'équilibre thermique et aérodynamique de la station, compromettant son efficacité.

Notre objectif est de renouveler l'air vicié grâce aux ventilateurs. Nous avons précédemment déterminé les paramètres environnementaux à observer et avons identifié l'indice de qualité de l'air AQI.

Nous souhaitons donc mettre en place un mode automatique pour lequel, lorsque l'AQI dépasse un seuil défini (par défaut AQI=50, mais celui-ci est ajustable dans le code ou à distance), le ventilateur s'active et sa puissance augmente linéairement entre deux valeurs limites de l'AQI. Ce mode automatique peut être amélioré afin de permettre un contrôle plus précis de la vitesse du moteur en fonction également de l'humidité et la température ou via des fonctions mathématiques avancées.

Dans certains cas, il peut être nécessaire d'activer ou d'intensifier la ventilation de manière ponctuelle. Pour cela, un mode manuel doit permettre aux opérateurs de contrôler directement la vitesse du ventilateur à distance.

3. L'environnement difficile d'une station de métro

3.1. Intégration du système aux infrastructures et alimentation

Nous souhaitons que notre système soit facilement intégrable dans des stations de métro ou des tunnels déjà existants et cela même dans des endroits isolés. Ainsi, il sera possible de

recupérer des données environnementales ou d'évacuer l'air vicié en tout point du métro sans contraintes liées aux infrastructures.

Pour cela, le système devra pouvoir se relier facilement au réseau électrique existant. Si une alimentation basse tension existe il est simple de s'y connecter, cependant il est également possible d'utiliser des systèmes à induction. Par exemple, l'entreprise islandaise Laki Power conçoit des systèmes de surveillance de lignes très haute tension qui sont alimentés par la tension sinusoïdale dans la ligne grâce à un système de bobines. Nous pouvons imaginer une alimentation du même type pour s'intégrer à l'infrastructure AC haute tension existante. Un système sur batterie est également envisageable.

Il est également nécessaire de mettre en place une communication à distance pour le contrôle des ventilateurs et pour faire remonter les données du capteur. Ainsi il sera possible de communiquer avec le matériel sans envoyer une équipe d'intervention et sans utiliser une communication filaire (onéreuse pour des systèmes isolés dans les tunnels).

3.2. Communiquer avec le système

3.2.1. La communication Lora

LoRa (*Long Range*) est une technologie de communication sans fil bas débit et longue portée, fonctionnant dans la bande de fréquence des ISM (Industrial, Scientific, Medical). Elle est particulièrement adaptée aux applications IoT nécessitant une faible consommation énergétique et une transmission de données sur de longues distances, même en milieu urbain ou souterrain. L'un des paramètres du LoRa sur lequel nous pourrions jouer est le SF (Spreading Factor) pouvant aller de SF7 à SF12. Nous choisirons SF=9 pour le reste du projet.

Nous utilisons le module RFM95W-868S2, qui fonctionne sur la bande de fréquence 868 MHz, adaptée à l'Europe. Il permet une longue portée avec une consommation d'énergie réduite, et est donc idéal pour notre application.

De plus, le module communique avec notre microcontrôleur via l'interface SPI car ce dernier est rapide et synchrone. Dans le cas de notre étude, une passerelle LoRaWAN est située à environ 100 mètres du dispositif, permettant de transmettre l'intégralité des données de nos capteurs. Cette communication avec la passerelle suit plusieurs étapes :

- **Joining** (Processus d'association au réseau) : l'appareil LoRa envoie une demande de connexion à la passerelle, puis cette dernière relaie cette demande au serveur réseau.
- **Join** (Association réussie et authentification) : une fois le périphérique accepté sur le réseau, il reçoit une clé de session lui permettant de chiffrer et d'envoyer ses données.
- **Uplink** (Envoi de données vers la passerelle) : les mesures des capteurs (température, CO₂, pression) sont envoyées périodiquement via LoRa vers la passerelle, qui les transmet au serveur.
- **Downlink** (Réception d'ordres envoyés depuis la passerelle) : dans notre cas, un downlink est utilisé pour activer un ventilateur sur demande de l'utilisateur. L'ordre de commande est envoyé via Datacake, transite par The Things Network (aussi abrégé TTN) puis est relayé par la passerelle vers le module, qui l'exécute

3.2.2. Communiquer sous la terre

Mais peut-on réellement utiliser LoRa dans un environnement aussi contraignant que le métro, où l'atténuation des ondes causée par la présence de tunnels épais et d'obstacles métalliques, les courbures du tunnel ou encore la présence de convois, pourrait sembler rédhibitoire. Plusieurs études montrent que LoRa demeure une solution viable, même dans ces conditions.

Une expérience menée dans le métro de Lisbonne a démontré que la communication LoRa peut s'établir entre deux stations, malgré l'épaisseur des galeries [8].

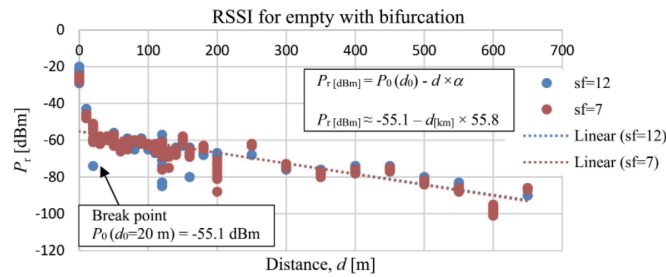


figure 6: Valeurs RSSI mesurées le long du tunnel pour SF=7 (rouge) et SF=12 (bleu)

Cette étude a permis de valider un modèle à deux pentes pour estimer l'atténuation du signal dans les tunnels. Les résultats présents dans le graphique ci-dessus montrent que :

- Pour les tunnels droits ou légèrement courbés, le facteur de perte est de -55,8 dB/km, permettant une portée de communication supérieure à 1,2 km (avec SF=12 et une sensibilité du récepteur de -132 dBm).
- Pour les tunnels courbés, le facteur de perte augmente à -80,5 dB/km, réduisant la portée maximale à environ 1 km dans les conditions les plus optimistes.

En outre, la présence d'un convoi dans le tunnel entraîne une atténuation supplémentaire d'environ 10 dB. Des tests supplémentaires sont nécessaires pour évaluer l'impact de deux convois se croisant dans un tunnel. Nous pourrions donc à priori bien communiquer entre plusieurs stations.

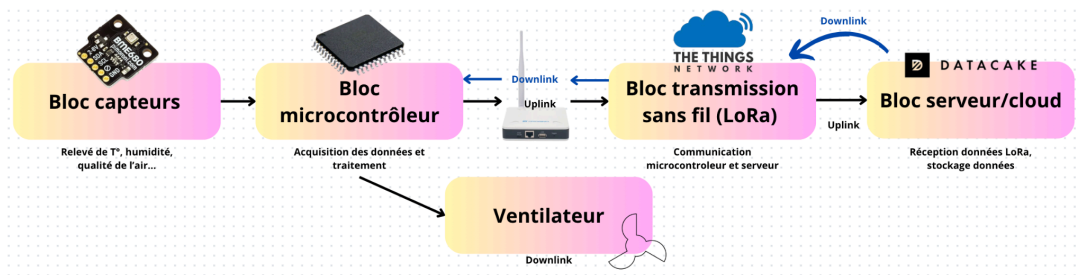
Par ailleurs, le projet du Grand Paris prévoit l'installation de points de connexion LoRa sur les nouvelles lignes de métro [10].

4. Conception du système

4.1. Synoptique

Notre système IoT se compose de plusieurs éléments interconnectés au sein de la plateforme OCASS :

- **Bloc capteurs** : regroupe les différents capteurs environnementaux (particules fines, CO₂, humidité, température).
- **Bloc microcontrôleur** : gère l'acquisition des données et la communication avec les autres modules.
- **Bloc transmission sans fil** : assure la liaison LoRa pour l'envoi des données vers une passerelle.
- **Bloc serveur/cloud** : stocke et visualise les données collectées via la plateforme Datacake.
- **Bloc ventilateur**



4.2. Description matérielle du module

Nous venons de voir que notre système était composé de plusieurs blocs fonctionnels. Dans cette idée une carte modulable permettant d'ajouter différents capteurs, moyens de transmission et actionneurs autour d'un microcontrôleur est intéressante. C'est le cas de la carte OCASS basée sur une puce STM32 et modulable avec l'ajout possible de 4 modules comprenant : un capteur de température, un capteur de qualité de l'air avec différents modules (nous utiliserons le Bosch BME680), une antenne mais aussi adaptable pour un capteur de fréquence cardiaque et facilement adaptable pour de nombreux cas d'application.

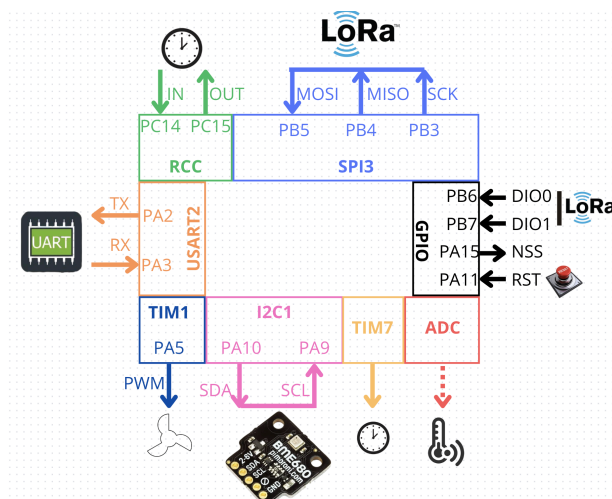


figure 8 : Pinout complet du système

Nous utilisons un ST-Link pour programmer et déboguer notre microcontrôleur. Celui-ci est une interface reliée à la carte via une nappe exploitant l'interface SWD (Serial Wire Debug). Contrairement au JTAG, qui fonctionne sur un bus parallèle avec plusieurs lignes de données, SWD utilise seulement deux fils (SWDIO et SWCLK). Il sert également pour l'alimentation même si nous utiliserons une batterie 9V pour notre maquette.

D'autre part, les cartes des capteurs ne permettent pas de debugger directement. Une des cartes de la plateforme permet d'utiliser facilement l'UART1 pour afficher des informations directement dans le terminal série. Lors de l'intégration de la carte air monitoring, cette connexion n'est plus disponible ce qui nous oblige à utiliser l'UART2 en se câblant sur des pins à droite de la core Layer (TX sur PA2 et RX sur PA3).

Enfin, les timers de la carte utilisés sont les suivants :

- Timer 1 : génération d'un signal PWM, qui contrôlera le ventilateur chargé d'améliorer la circulation de l'air dans la station. Ce fonctionnement est analogue à celui d'un moteur et sera détaillé dans une partie ultérieure.
- Timer 7 : utilisé comme source d'horloge principale du système. Il cadencera les différentes tâches et permettra de synchroniser la récupération et l'envoi des données. Les différentes tâches sont lancées via les fonctions os comme `os_setTimedCallback` qui indique un lancement après n secondes.

Enfin, la carte et ses capteurs fonctionnent sous 3.3V, tandis que certains actionneurs comme le ventilateur nécessitent une tension de 12V.

4.3. Mise en place de la communication LoRa

Notre objectif est de faire remonter les données de la carte OCASS vers une gateway Lora à partir de laquelle les données seront déposées sur une plateforme en ligne : TheThingsNetwork afin d'être traitées.

4.3.1. Code

Pour la transmission nous utilisons Cayenne qui est une plateforme IoT développée par myDevices. Elle permet d'avoir un format de données spécifique appelé Cayenne Low Power Payload (Cayenne LPP) pour standardiser les messages envoyés via LoRaWAN. Nous pouvons ensuite traiter plus facilement les données en se basant sur le standard CayenneLPP. Il a été développé pour permettre l'envoi de données avec une consommation minimale d'énergie et une empreinte mémoire réduite. LPP encode chaque mesure sous forme de trames structurées comprenant un numéro de canal, un type de donnée et une valeur codée sur un nombre fixe d'octets. Un tableau est disponible en annexe détaillant les types existants.

Pour utiliser ce standard nous disposons de la librairie `cayennelpp.h` qui nous donne accès à différentes fonctions pour remplir un buffer contenant les trames à envoyer avec le type de données et la valeur. Le buffer est défini via la structure `cayenne_lpp_t`.

Pour ensuite transférer les données sur Datacake, il est nécessaire de paramétrer la plateforme TheThingsNetwork pour récupérer des trames à ce format ce qui nécessite uniquement un réglage dans l'onglet "Payload Formatters".

Pour la transmission LoraWan nous utilisons ensuite une autre bibliothèque. LMIC (LoRaMAC-in-C) est une bibliothèque open-source en langage C qui implémente la pile LoRaWAN. Elle permet de gérer les protocoles de connexion, l'envoi et la réception de messages. Nous nous basons sur le code déjà donné afin d'effectuer les différentes étapes de connexion déjà données dans la partie Lorawan de ce document. Pour l'envoi nous utilisons enfin la fonction suivante :

```
LMIC_setTxData2(1, &lpp, nb_bytes, 0);
```

`nb_bytes` est le nombre d'octets à envoyer en fonction des données que nous avons précédemment ajoutées dans le buffer `lpp` via les fonctions `cayenne_lpp_add_XXX` où XXX est le type de données (`analog_output`, `temperature`, `luminosity`...).

Le 0 indique que nous sommes en mode non confirmé (aucune réponse du serveur requise).

4.3.2. Antenne

Les données sont ensuite envoyées à l'antenne, via une connexion SPI qui a pour avantage principal d'opérer à une grande vitesse. Le module de transmission sans fil sur la carte OCASS va ensuite moduler les données pour préparer leur transmission dans des ondes électromagnétiques. La porteuse est à 868MHz sur une bande publique. Ce signal crée un

champ électromagnétique oscillant. L'antenne, qui est un conducteur dimensionné spécifiquement pour cette fréquence (environ 17 cm de long pour le 868 MHz), rayonne alors ces ondes électromagnétiques dans l'espace. Enfin la réception se fait via une autre antenne de taille similaire sur la gateway Lora.

L'antenne et le module de communication sans fil de la carte OCASS peuvent également être utilisés en réception pour des données envoyées via un Downlink.

4.4. Intégration de la carte air monitoring

L'intégration de la carte "air monitoring" à la plateforme OCASS nécessite la suppression du PCB CONN1, qui embarquait l'UART1, une LED ainsi qu'un capteur de température. Cette modification libère l'espace nécessaire pour connecter la carte de surveillance de l'air, qui apporte de nouvelles fonctionnalités, notamment la mesure de la qualité de l'air via le capteur BME680, qui communique avec le microcontrôleur via le bus I2C.

De plus, nous avons remarqué qu'en l'absence de calibration de ce dernier, les mesures étaient parfois peu pertinentes voire absurdes. La calibration se fera donc dès le démarrage du programme via la fonction suivante :

```
bme68x_start(&data, &hi2c1); // Initialisation et calibration du capteur BME680
```

Enfin, les relevés sont effectués via la fonction "single measure", qui remplit la structure bme68x_data avec l'ensemble des paramètres mesurés, dont voici les principales données retournées :

```
uint8_t status; //etat capteur (dispo pour mesure ou non)
uint8_t gas_index; //profil de chauffage pour mesure gaz
uint8_t meas_index; // numéro de la mesure dans la séquence d'acquisition
uint8_t res_heat; //résistance chauffage interne (mesure gaz)
uint8_t idac; //courant DAC
uint8_t gas_wait; //temps d'attente avant mesure gaz
float iaq_score; //indice AQI
```

4.5. Interface de contrôle et visualisation via Datacake

Pour faciliter l'utilisation de la plateforme à distance, nous avons mis en place une interface de visualisation et de contrôle sur Datacake. Cette plateforme permet entre autres d'afficher en temps réel les mesures des capteurs et d'envoyer des commandes au système via des downlinks. Pour cela, on crée un appareil qui est ensuite configuré pour récupérer les données envoyées par notre module via TTN (on choisira à cet effet la version TTN V3 compatible avec notre système).

Une fois l'appareil configuré, les données envoyées par les capteurs (CO₂, humidité, température) sont automatiquement affichées sur un dashboard. En plus de la visualisation, Datacake permet d'envoyer des commandes à distance au système via les downlinks. Par exemple, l'opérateur pourra activer/désactiver le mode automatique du ventilateur ou régler manuellement la puissance du ventilateur. Nous avons paramétré le système pour qu'il soit utilisable en ligne par les personnes possédant l'URL. Datacake permet en effet de donner un accès libre à l'interface ou d'en sécuriser l'accès via un mot de passe. Notre système est accessible via : <https://app.datacake.de/pd/4ff97c3b-e6d7-4391-8f15-8dbe7441a91a>

4.6. Downlink et intégration d'un ventilateur

Jusqu'ici nous avons uniquement intégré la remontée d'informations depuis la carte OCASS (Uplink) afin de récupérer les données des capteurs. Nous souhaitons maintenant intégrer un ventilateur sur la carte qui sera contrôlé directement via le microprocesseur mais pourra également être dirigé à distance.

4.6.1. Mise en place du moteur

Nous utilisons un ventilateur turbine 12V récupéré dans un stock de pièces détachées dans l'association de robotique de l'école des Mines. Nous souhaitons piloter sa vitesse, pour cela nous devons mettre en place un contrôle via PWM. Les sorties de la carte ne peuvent pas nous donner la tension et le courant nécessaires pour alimenter le moteur. Nous allons donc utiliser un hacheur que nous commanderons via une PWM provenant d'un timer de la STM32. Nous avons uniquement une batterie 9V à disposition dans notre prototype, le moteur ne pourra donc pas fonctionner à pleine vitesse.

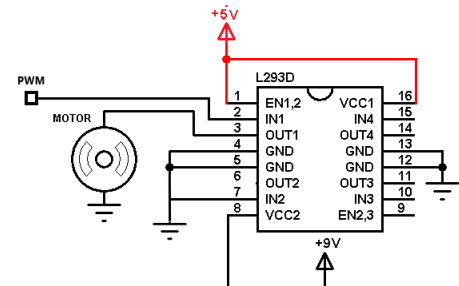


figure 9 : Schéma d'alimentation du moteur

Nous câblons donc un driver pour moteur DC (le L293D qui est le plus utilisé pour la mise en place rapide de moteurs à courant continu). Nous utilisons le schéma de câblage ci-contre qui nous permet de choisir la vitesse via une PWM. Le sens est cependant imposé par le câblage puisque la turbine a un seul sens de rotation.

Nous commandons maintenant le moteur via une PWM à 500Hz. Pour cela nous utilisons le timer 1 CH1 pour lequel nous calculons comme paramètres PSC opti = 1 et ARR = 160 000. Pour vérifier son bon fonctionnement, nous visualisons le signal à l'oscilloscope.

Nous observons via des tests que le moteur commence à tourner à partir de 4V. Nous mettons donc en place une fonction dans le code qui détermine la vitesse du moteur en fonction d'un pourcentage entre 0 et 100. La subtilité est que à 0% le moteur ne tourne pas donc la PWM est à 0 mais à partir de 1% le moteur doit tourner au minimum de sa vitesse donc la valeur moyenne de la PWM doit être 4V. Nous utilisons donc l'algorithme ci-dessous pour donner la valeur de l'ARR du moteur.

```
Algorithme valueToSpeed
Entrées :
- valeur : entier (valeur en pourcentage)
- pourcentageVitesseMin : entier (alimentation minimale en pourcentage de l'alim)
Sortie :
- vitesse : entier (valeur convertie en vitesse PWM)

Début
  minVitesse ← max_vitesse_compteur * (pourcentageVitesseMin / 100.0)
  vitesse ← valeur * (max_vitesse_compteur - minVitesse) / 100 + minVitesse
  Afficher(vitesse)
  Retourner vitesse
Fin
```

4.6.2. Contrôle du ventilateur

Nous mettons en place 2 modes de contrôle pour le ventilateur, le mode automatique et le mode manuel. Par défaut, le mode automatique est activé ce qui fait que le moteur s'active

lorsque l'AQI dépasse un seuil (initialement fixé à 50) et prend une vitesse proportionnelle à l'AQI (entre la valeur minimale et une valeur maximale également donnée). L'algorithme de contrôle automatique est donné en annexe de ce document.

Nous souhaitons que l'opérateur à distance puisse activer ou désactiver le mode automatique et choisir la vitesse. Pour cela il utilise un Downlink depuis Datacake. Un downlink prend de la bande passante, nous souhaitons donc minimiser la quantité de données envoyée. Nous souhaitons transférer une valeur entre 0 et 100 pour la puissance du moteur et un booléen caractérisant l'activation du mode automatique.

L'opérateur choisit donc les paramètres via l'interface ci-contre puis un Downlink est envoyé lorsqu'il clique sur "Send" avec comme bit de poids fort le booléen caractérisant l'activation du mode automatique et pour les 7 bits de poids faible la vitesse du moteur (7 bits permettent de fixer une valeur entre 0 et 128 ce qui est suffisant ici).

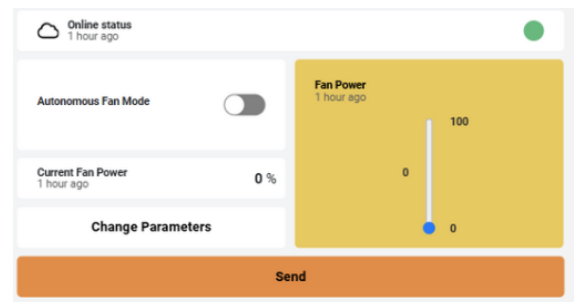


figure 10 : Widget ventilateur

Le code C permettant de préparer les données du Downlink pour l'envoi depuis Datacake est donné en annexe.

Nous testons ce Downlink et obtenons un fonctionnement parfait, la vitesse du ventilateur est mise à jour dès la réception qui a lieu quelques dizaines de secondes après l'envoi du Downlink. Le Downlink n'est donc pas adapté à un contrôle en temps réel.

Nous avons également mis en place un second Downlink pour changer les paramètres internes du microcontrôleur sans le reprogrammer. Dans notre cas, nous modifions le délai entre chaque transmission, la valeur minimale d'AQI pour activer le ventilateur et la valeur maximale correspondant à l'AQI pour lequel la puissance est maximale. Ceci est fait via un Downlink envoyant une trame de 4 octets avec les différentes informations et un "identifiant de trame" permettant de la différencier d'une trame de contrôle du ventilateur. Le code d'envoi du Downlink et des détails concernant ce Downlink sont disponibles en annexe.

5. Test du système

5.1. Conception d'une maquette

Pour démontrer le fonctionnement de notre système dans un environnement "réaliste", nous avons conçu une maquette simulant une station de métro. L'ensemble des capteurs est placé à l'intérieur d'une boîte fermée, représentant la station.

Nous isolons également le système en l'alimentant via une batterie 9V avec la nappe de connexion au ST-Link déconnectée mais passant par un orifice pour pouvoir déboguer via la connexion UART. Cela permet d'illustrer la possibilité d'un système entièrement embarqué, sans connexion à un ordinateur. Le ventilateur, dont l'alimentation est également assurée par la pile 9V, permet d'évacuer l'air vicié est intégré à la structure. Il est fixé sur la paroi supérieure avec la sortie à l'extérieur pour une aspiration de l'air à l'intérieur de la boîte. Le ventilateur est piloté via un driver connecté sur une breadboard également à l'intérieur de la boîte.



figure 11 : Prototype

Cette maquette nous permet ainsi de tester l'efficacité de notre dispositif. En effet, nous pouvons insuffler du CO₂ à l'intérieur de la boîte via un trou prévu à cet effet et le CO₂ restera

bloqué à l'intérieur de la boîte (qui n'est pas étanche mais limite sa dissipation dans l'air et permet de simuler un milieu confiné).

5.2. Interprétation des résultats

5.2.1. Analyse perf

Nous obtenons les résultats suivants sur nos widgets :

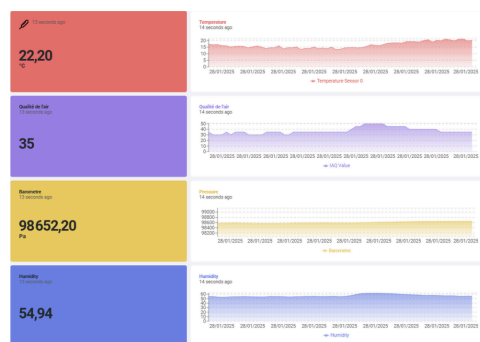


figure 12 : Widgets Datacake

Nous avons donc un accès rapide à l'information. Tous les différents éléments fonctionnent parfaitement et les données sont remontées précisément. Nous avons pu laisser tourner le système pendant longtemps et avons obtenu des résultats cohérents et stables. Le diagramme de qualité de l'air ci-dessous montre bien une hausse de la "pollution" correspondant à l'injection de CO2 dans la boîte (en soufflant)

Lorsque nous utilisons le prototype nous observons que souffler à l'intérieur fait bien augmenter la valeur de la qualité de l'air.

De plus, le système de ventilateurs s'active parfaitement lors du dépassement du seuil et les downlinks depuis Datacake sont pleinement fonctionnels.

5.2.2. Actions

Par ailleurs, lors de la phase de test, nous avons validé le bon fonctionnement des deux types de downlinks : l'activation/désactivation du mode automatique et le réglage manuel de la vitesse du ventilateur. Les tests ont montré que les commandes envoyées via Datacake étaient bien reçues et interprétées par notre prototype.

Enfin, afin d'évaluer l'impact du ventilateur sur la qualité de l'air, nous avons réalisé un test comparatif en mesurant l'évolution de l'AQI avec et sans ventilation. Le graphique ci-dessous illustre cette différence, mettant en évidence l'efficacité du ventilateur pour accélérer la purification de l'air :

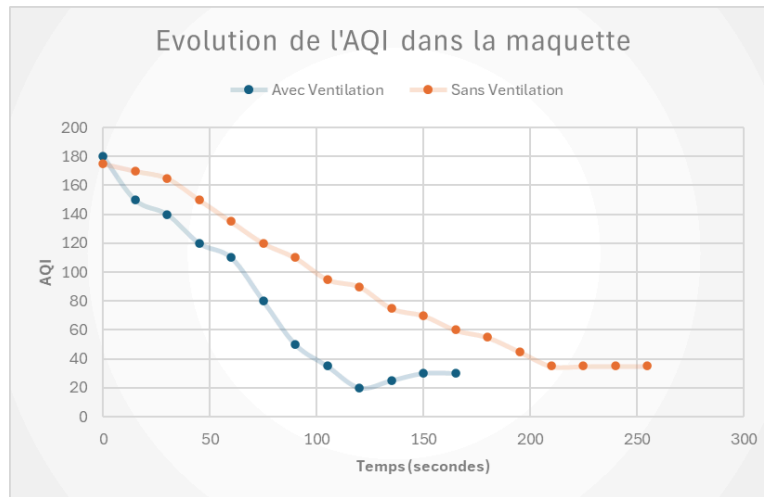


figure 13 : Évolution de l'AQI avec et sans ventilation

5.3. Limites observées

5.3.1. Consommation énergétique

Un des premiers paramètres limitants est la consommation électrique du système. Nous n'avons pas pu explorer cet aspect mais l'économie d'énergie n'a pas été prise en compte lors de la conception de notre code embarqué et dans le choix des capteurs utilisés. Ainsi, nous avons tout d'abord remarqué qu'une calibration était nécessaire pour le capteur BME ce qui consomme de l'énergie à l'initialisation. La communication via l'antenne est également consommatrice d'énergie. Il serait donc important de minimiser le nombre de données envoyées. Actuellement nous envoyons 8 octets de données (4 fois 2 octets) nous pourrions minimiser les données envoyées pour économiser de l'énergie. Espacer l'envoi des trames est également envisageable.

De plus, l'utilisation du mode Low Power serait intéressante pour mettre en veille le système entre les envois puisque nous effectuons la mesure uniquement avant l'envoi. De plus, nous effectuons des relevés de tous les indicateurs. Ainsi, l'AQI est calculé à partir de la résistance du gaz et d'autres paramètres. Ce calcul est complexe et consomme de l'énergie. Il pourrait ainsi être intéressant de transmettre les valeurs mesurées brutes avant calcul pour effectuer un calcul sur un système dont la consommation n'est pas limitée.

5.3.2. Qualité du réseau Lora

Bien que LoRa soit conçu pour de longues distances, nous avons constaté que la connexion dépendait fortement de la gateway notamment si celle-ci est en intérieur. Pourtant située à seulement une centaine de mètres, cette dernière s'est parfois montrée capricieuse, mettant un certain temps à accepter le join du module. Cette instabilité, possiblement due à la congestion du réseau ou à des paramètres de configuration, montre que LoRa reste tributaire de l'infrastructure en place. La communication Lora ne permet pas non plus un contrôle en temps réel via des Downlinks et oblige à rester en dessous de 1% d'occupation du réseau.

5.4. Améliorations potentielles

5.4.1. Techniques

Afin d'améliorer le système, nous pourrions mener à bien les améliorations suivantes. Premièrement, nous pourrions ajuster dynamiquement le SF dans le but d'optimiser le compromis entre portée et consommation d'énergie, en réduisant le SF lorsque le signal est bon et en l'augmentant lorsque la connexion est plus faible.

D'autre part, nous pourrions instaurer un mécanisme de reconnexion automatique en cas de perte de connexion avec le réseau LoRaWAN. Un rejoin automatique permettrait d'assurer une reprise des communications.

5.4.2. Régulation du trafic et des passagers

Par ailleurs, une gestion plus fine du ventilateur pourrait être implémentée, en intégrant davantage de paramètres comme l'affluence dans la station. Par exemple, des capteurs de présence ou des données externes pourraient permettre d'ajuster la vitesse du ventilateur non seulement en fonction de la pollution, mais aussi de la densité de personnes présentes

Au-delà des mesures de ventilation et de filtration, la gestion des flux de passagers pourrait constituer un levier supplémentaire pour limiter l'exposition aux pics de pollution et donc une perspective assez intéressante pour le projet. L'intégration de notre système à des applications de mobilité comme BonjourRATP pourrait permettre d'alerter en temps réel les usagers sur les niveaux de pollution dans les différentes stations. Par exemple, lorsqu'une station atteint un seuil critique de particules fines ou de CO₂, une notification pourrait être envoyée aux voyageurs pour leur suggérer une alternative, comme une ligne moins encombrée ou un itinéraire combinant plusieurs modes de transport (bus, tram, marche à pied...).

Enfin, l'intégration des données de nos capteurs avec celles d'OpenRATP et AirParif permettrait d'entraîner un modèle IA capable d'anticiper les pics de pollution. Cela optimiserait la gestion du ventilateur et permettrait d'alerter les usagers en amont.

6. Conclusion

Ce projet, nous a permis d'aborder une problématique concrète liée à la qualité de l'air dans les stations de métro et d'explorer des solutions techniques applicables au monde réel. Nous avons ainsi pu voir une application du travail d'ingénieur pour le développement d'un outil améliorant concrètement la santé des citoyens. Bien qu'il s'agisse d'un travail scolaire, nous avons cherché à concevoir un système fonctionnel en nous appuyant sur des technologies telles que les capteurs environnementaux, la communication LoRa et le contrôle automatique d'un ventilateur. LA plateforme OCASS a été une force dans ce projet puisqu'elle nous a permis de concevoir une carte sur mesure utilisant uniquement les capteurs et l'antenne et se basant sur un microcontrôleur que nous avons déjà utilisé dans le passé et qui permet d'utiliser différents périphériques.

La réalisation d'une maquette a été une étape intéressante du projet, nous permettant de simuler les conditions d'un environnement confiné et de tester l'efficacité de notre dispositif. Grâce à cette maquette, nous avons pu observer en temps réel l'impact du ventilateur sur la qualité de l'air et valider le bon fonctionnement du système, notamment la transmission des données et le contrôle à distance via Datacake. Ces expérimentations nous ont permis d'identifier des points d'amélioration (par exemple de voir qu'un contrôle à distance était intéressant) et d'affiner notre approche.

Ce projet nous a également amenés à réfléchir aux défis d'une intégration d'un système IOT dans un système de transport existant. Si notre prototype reste une preuve de concept, il ouvre la voie à des applications concrètes et pourrait être amélioré pour répondre aux contraintes opérationnelles d'un déploiement à grande échelle. De plus, ce projet est en lien avec des solutions déjà existantes comme l'intégration des capteurs communiquant en Lora dans le cadre du projet Grand Paris Metro. En conclusion, cette expérience nous a offert l'opportunité d'allier la théorie acquise et pratique, en développant une solution innovante et

en nous confrontant aux défis techniques d'un projet IoT appliqué à un enjeu environnemental majeur.

Bibliographie et sources

- [1] OpenData RATP, "Explore RATP data". Disponible : <https://data.ratp.fr/explore/?sort=modified>
- [2] France Télévisions, "Enquête France TV : Pollution dans le métro parisien, découvrez les mesures inédites de la qualité de l'air dans votre station". Disponible : https://www.francetvinfo.fr/monde/environnement/crise-climatique/pollution-air/enquete-france-tv-pollution-dans-le-metro-parisien-decouvrez-les-mesures-inedites-de-la-qualite-de-l-air-dans-votre-station_5815343.html
- [3] Airparif, "Stations de métro et de RER". Disponible : <https://www.airparif.fr/stations-de-metro-et-de-rer>
- [4] Île-de-France Mobilités, "Qualité de l'air dans le réseau de transport francilien". Disponible : <https://data.iledefrance-mobilites.fr/explore/dataset/qualite-de-lair-dans-le-reseau-de-transport-francilien/information/>
- [5] Le Monde, "Pollution de l'air dans le métro : une étude alerte sur le niveau de particules fines", *Le Monde*, 23 mai 2023. Disponible : https://www.lemonde.fr/planete/article/2023/05/23/pollution-de-l-air-dans-le-metro-une-etude-alerte-sur-le-niveau-de-particules-fines_6174442_3244.html
- [6] A. Marques, *Système À Microcontrôleur 2*, Version VP2024 – V2.
- [7] A. Marques, *LoRa*
- [8] C. Sousa, C. Alves, J. Casaleiro, and A. Serrador, "An IoT Radio Coverage Survey in Subway Tunnels," *2023 IEEE International Conference on Internet of Things (ICIOT)*, 2023, pp. 1-5. [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10292809>.
- [9] J. Smith, L. Doe, and R. Lee, "Performance Analysis of LoRa in Urban Subway Environments," *2023 IEEE Global Communications Conference (GLOBECOM)*, 2023, pp. 6-10. [Online]. Available: <https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=10065792>.
- [10] "Kerlink helps Grand Paris Metro project realize tunnel vision with LoRaWAN connectivity," *IoT Now*, Jun. 28, 2018. [Online]. Available: <https://www.iot-now.com/2018/06/28/84941-kerlink-helps-grand-paris-metro-project-realize-tunnel-vision-lorawan-connectivity/>.

Annexes

Types de trames CayenneLPP

Type	IPSO	LPP	Hex	Data Size	Data Resolution per bit
Digital Input	3200	0	0	1	1
Digital Output	3201	1	1	1	1
Analog Input	3202	2	2	2	0.01 Signed
Analog Output	3203	3	3	2	0.01 Signed
Illuminance Sensor	3301	101	65	2	1 Lux Unsigned MSB
Presence Sensor	3302	102	66	1	1
Temperature Sensor	3303	103	67	2	0.1 °C Signed MSB
Humidity Sensor	3304	104	68	1	0.5 % Unsigned
Accelerometer	3313	113	71	6	0.001 G Signed MSB per axis
Barometer	3315	115	73	2	0.1 hPa Unsigned MSB
Gyrometer	3334	134	86	6	0.01 °/s Signed MSB per axis
GPS Location	3336	136	88	9	Latitude : 0.0001 ° Signed MSB
					Longitude : 0.0001 ° Signed MSB
					Altitude : 0.01 meter Signed MSB

Contrôle automatique du moteur

```
Algorithme ConvertirIAQenVitesse
Entrées :
- minIAQ : entier (valeur minimale de l'IAQ)
- maxIAQ : entier (valeur maximale de l'IAQ)
- pourcentageVitesseMin : entier (pourcentage de la vitesse minimale)
Sortie :
- vitesse : entier (valeur convertie en vitesse PWM)

Début
  IAQ ← data.iaq_score

  Si IAQ < minIAQ Alors
    Retourner 0
  Fin Si

  Si IAQ > maxIAQ Alors
    Retourner max_vitesse_compteur
  Fin Si

  pourcentage ← (IAQ - minIAQ) * 100.0 / (maxIAQ - minIAQ)
  puissance_ventilateur_autonome ← pourcentage

  vitesse ← ConvertirValeurEnVitesse(pourcentage, pourcentageVitesseMin)
  Retourner vitesse
Fin
```

Encodeur du Downlink de contrôle du moteur

```
function Encoder(measurements, port) {
  // Return a byte array here
  var fan = measurements["FAN_POWER"].value;
```

```

fan = (fan & ~(1 << 7)) | (measurements["AUTONOMOUS_FAN_MODE"].value << 7);
return [fan];
}

```

Encodeur du Downlink de changement des paramètres

```

function Encoder(measurements, port) {
  var delay = measurements["DELAY_BETWEEN_TRANSMISSION"].value;
  var min_IAQ = measurements["MIN_IAQ"].value / 5;
  var max_IAQ = measurements["MAX_IAQ"].value / 5;
  return [0xFF, delay, min_IAQ, max_IAQ];
}

```

NB : min_IAQ sont divisés par 5 afin de pouvoir transmettre une valeur entre 0 et 500 sur un seul octet (nécessitant des valeurs entre 0 et 256).

La valeur 0xFF correspond à "l'identificateur de la trame". Ainsi, dans le code du microcontrôleur nous pouvons identifier qu'il s'agit d'un Downlink pour le changement des paramètres. La valeur 0xFF a été choisie car elle ne peut pas être confondue avec un Downlink de contrôle des moteurs puisque la valeur maximum prise dans un contrôle du moteur est 228 ce qui correspond à l'activation du mode automatique (bit de poids fort à 1) et une vitesse à 100. Nous pouvons donc faire la différence entre les 2 types de trame dans le code sans risque.